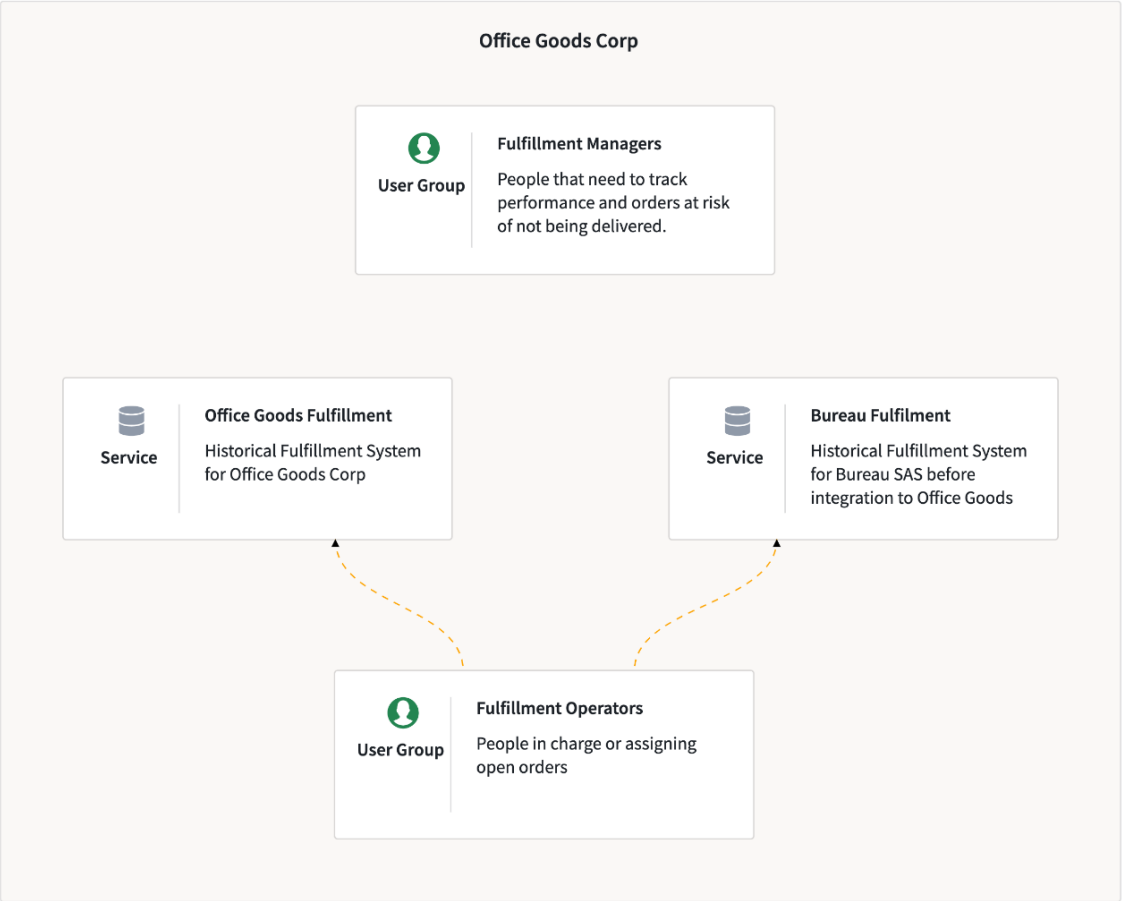
Your First E2E Workflow

Overview

The Use Case

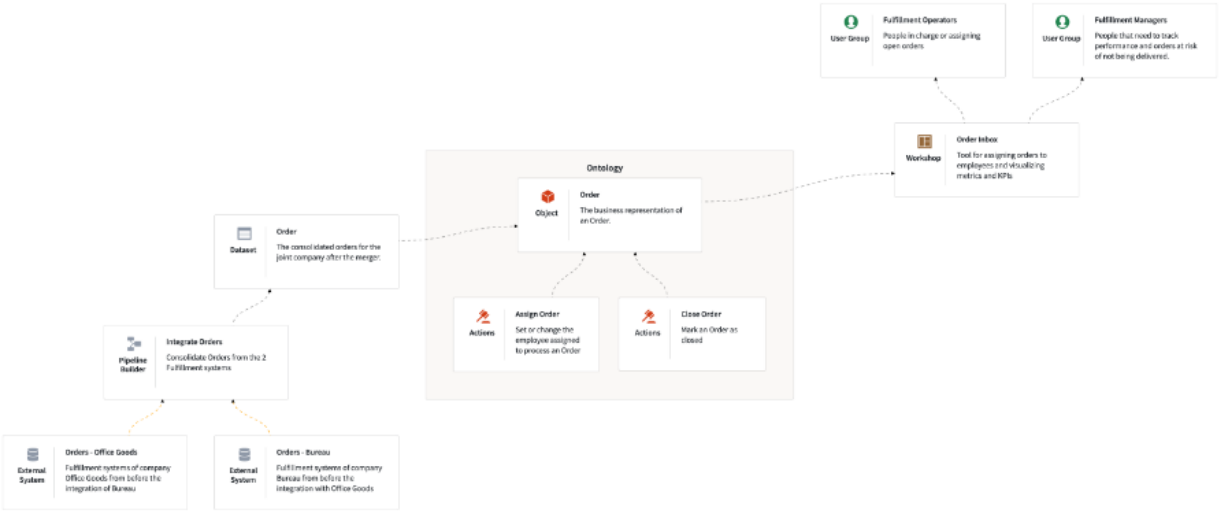
You work for **Office Goods Corp**, a paper and office goods company. Recently, your company acquired one of its competitors, **Bureau SAS.** The IT unification project won’t start for another year, which means that your colleagues are stuck tracking and managing orders on two different systems.

They've survived a couple months by manually stitching spreadsheet, but with demand growing, things have started to fall through the cracks. Incorrect metrics, unfulfilled orders, and unhappy customers. The fulfillment department has reached out to you for help, to see what can be done quickly ahead of the unification project.



After understanding the problem a little better, you have decided to create a simple operational tool for the fulfillment department that connects to both IT systems. It will create a single location for managers to assign orders, show what orders haven’t been assigned, and what is at risk of not being delivered. And along the way end debates about which spreadsheet is the most up-to-date!

You've sketched together the below diagram showing what you're going to build, and how it's going to work.



Introduction

All of the things you create in Foundry need to live inside a Project, so let’s first start by creating a Project. If you already have a project you can use, feel free to skip to the “Create Folder” step. If you don’t have a project, and don’t have the permissions to create one, please contact your platform administrator or Palantir representative.

Introduction

Foundry includes a powerful **Data Connection** application. It comes with connectors for over 200 systems and databases and a flexible plugin architecture for the rest. It also has powerful DevOps tools for managing recurring batch and streaming **ingestions**. However, for the sake of this tutorial, we will drag-and-drop the data we need.

You should have access to three files:

1. orders_office_goods.xlsx: All the orders from Office Goods Corp., the fictional company you work for
2. orders_bureau_transactional_system.xlsx: All the orders from Bureau SAS, the fictional company your company bought
3. consolidated_customers.xlsx: A consolidated list of customers across both fictional companies

Downloads

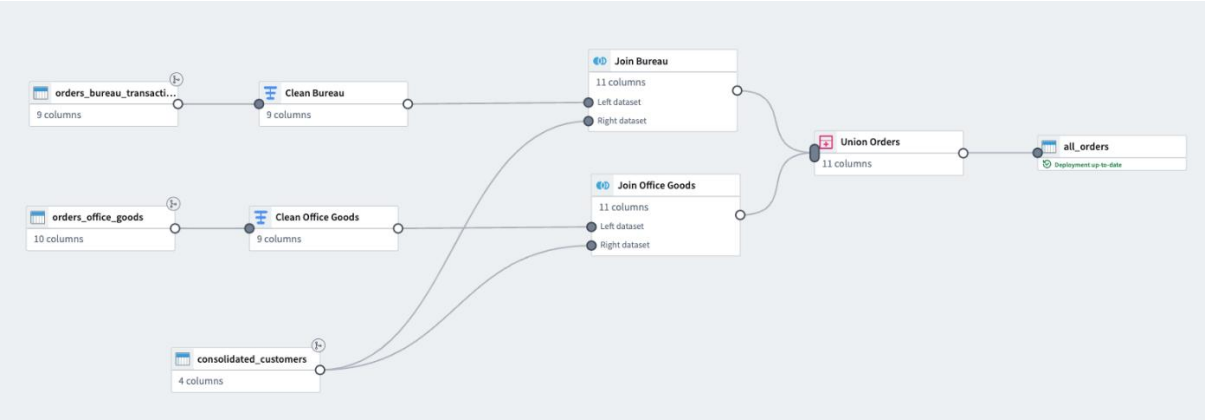
- consolidated_customers.xlsx
- orders_bureau_transactional_system.xlsx
- orders_office_goods.xlsx

Introduction

In the next steps, we are going to clean the datasets, join them with the master customer list, and union them so that we have a single dataset containing all orders.

Foundry has several ways to do this. In this tutorial, we will use **Pipeline Builder**, which is a flexible low-code/no-code app for rapidly prototyping and building production-grade pipelines.

By the end of this section, you will have created the following pipeline and outputs:



Introduction

We now need to make the all_orders dataset expressive and queryable, which is done through the **Ontology**. The Ontology is a relational “digital twin” of your business comprised of **Objects**. It’s designed to evolve quickly, since your business evolves quickly, too!

The Ontology is a compounding asset: Once you build the Objects for one usecase, you can reuse them for other usecases and avoid reinventing the wheel. You can even create API bindings so that your applications outside of Foundry can interact with the Ontology in a first-class way.

We will define the Order object by using the **Ontology Manager App**.

Introduction

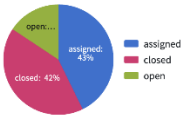
The next step is to create a tool that solves the Fulfillment Department's problems. **Workshop** is a tool in Foundry for rapidly prototyping and creating powerful applications that leverage the Ontology.

By the end of this section, you will have created a classic dashboard with basic KPIs, charts, and filtering capabilities. Later on, we will make this interactive and collaborative by allowing you to modify who is assigned to an order.

(your username) Orders Inbox

Call-To-Action Button

Charts



All Orders

Item Name	Assignee	Consolidated Customer Id	Customer Id	Customer Name	Days Until Due	Order Due Date	Order Id	Q
A1	*No value*	6a0f29f5-5c51-479f-a1de-	A16737082	ACME Consolidated	53	Jul 24, 2023, 2:00 AM	A37897985-2876d968-98e4-	24
A1	Gail Weber	eabbfb77-391b-4a66-94cf-	27564272-14f6-4456-9e26-	Hessel - Muller	8	Jun 9, 2023, 2:00 AM	2c32e4bd-5938-469d-9e0f-	29
A1	Latoya Gulgowski	b38352f4-2e78-4530-a94f-	A41637514	King, Feeney and Kutch	42	Jul 13, 2023, 2:00 AM	A37328113-2876d968-98e4-	53
A1	Alfredo Bins	672d3a1d-d7c2-4fca-bf22-	A51330122	Haley, Hackett and Torp	43	Jul 14, 2023, 2:00 AM	A73029460-2876d968-98e4-	27
A1	Kristen Mohr	ca79e915-18af-43da-824f-	A88692123	Kris - Jakubowski	9	Jun 10, 2023, 2:00 AM	A2494962-2876d968-98e4-	39
A1	Lela Kemmer	891dffa3-3e6b-4ad5-b5b5-	39defc1b-a98e-421b-a221-	Dibbert Inc	19	Jun 20, 2023, 2:00 AM	cd66f4bd-326b-452b-b3d8-	59
A1	Gail Weber	ca79e915-18af-43da-824f-	185bc542-47d7-4a00-9bbe-	Kris - Jakubowski	36	Jul 7, 2023, 2:00 AM	f3ec8140-bbe9-4ed8-b522-	74
A1	*No value*	942bef52-5a85-4110-b887-	A46828545	Feil Group	20	Jun 21, 2023, 2:00 AM	A52333958-2876d968-98e4-	13
A1	John Dooley	f9d48f25-46b6-4c8f-bc0a-	A86993846	Buckridge Inc	23	Jun 24, 2023, 2:00 AM	A4620333-2876d968-98e4-	33
A1	Kristen Mohr	6bf2afa9-1cda-484f-8dc8-	A56613925	Watsica, Turner and Toy	47	Jul 18, 2023, 2:00 AM	A98415914-2876d968-98e4-	46
A1	Erick Quigley	c3446c26-832c-4d3e-bd89-	5edd5d4a-52fb-46c6-9981-	Dare, Dare and Smith	15	Jun 16, 2023, 2:00 AM	f76941d2-2b35-43db-899f-	73
		bbfb5080-fa1a-		Marquardt -		Jun 28, 2023	A31854563-	

A1

View full details

Item Name	A1
Assignee	*No value*
Consolidated Custo...	6a0f29f5-5c51-479f-a1
Customer Id	A16737082
Customer Name	ACME Consolidated Eu
Days Until Due	53
Order Due Date	Jul 24, 2023, 2:00 AM
Order Id	A37897985-2876d968-
Quantity	24
Status	open

Filter (your username) Orders

ITEM NAME
Search...

ASSIGNEE
Search...

CONSOLIDATED CUSTOMER ID
Search...

CUSTOMER ID
Search...

CUSTOMER NAME
Search...

DAYS UNTIL DUE
-

ORDER DUE DATE
Start date | End date

ORDER ID
Search...

QUANTITY
-

STATUS
Search...

Introduction

We now want to make it possible to interact and edit the Ontology. We will do this by creating **Actions**. Actions are events and edits that are performed on an object or set of objects in the Ontology. For example, you might have an Action to set a status as closed, or to create a new Order, or delete an incorrect Order. You can also define API bindings for Actions so that external applications may also interact with the Ontology!

By the end of this section you will have created an “Assign Order” Action that performs two actions at once: It sets the assignee to whatever name you provide, and it automatically sets the Status to “assigned.”

Introduction

While read-only dashboards can be very useful, it’s important that multiple people and teams be able to collaborate, organize themselves, and update state. Otherwise we won’t have solved the Fulfillment Department’s problem!

You’ll see how easy this is by adding the “Assign Order” action to the Workshop application you’ve built.

You’ve now created an operational dashboard for the Fulfillment Department that serves as the single source of truth and cockpit about allocations for upcoming orders.

And it is relatively simple to make this even more useful and more secure:

- Enhance visualizations
- Send email notifications to users when assigned to an order
- Add comments section to provide status updates for each order
- Replace static data file imports with live feeds from the transactional systems
- Control who can assign and mark closed an order
- Automatically generate audit trails of who made a modification or status change
- Write back the actions (assignee, status) back to the transactional systems in real-time

You now have the basics for developing operational data applications!